

Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Кубанский государственный технологический
университет»

Кафедра информационных систем и программирования

СИСТЕМЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Методические указания по выполнению лабораторных работ
для студентов всех форм обучения направления подготовки

09.03.04 Программная инженерия

Краснодар

2023

Составитель: канд. техн. наук, доц. К.Е. Тотухов

Системы искусственного интеллекта: методические указания по выполнению лабораторных работ для студентов всех форм обучения направления подготовки 09.03.04 Программная инженерия / Сост.: К.Е. Тотухов; Кубан. гос. технол. ун-т. Каф. информационных систем и программирования. – Краснодар, 2023. – 23 с.

Содержат перечень лабораторных работ с указанием целей и задач их выполнения, теоретическую информацию к каждой лабораторной работе, варианты заданий, формы итогового контроля, ссылки на литературу.

Рецензенты: д-р техн. наук, ст. науч. сотр., проф. кафедры ИСиП КубГТУ
Л.А. Видовский;

канд. техн. наук, доц., доц. кафедры математики и выч. техники
Академии маркетинга и социально-информационных
технологий Н. С. Нестерова

Содержание

Лабораторная работа №1 Экспертные системы.....	4
Лабораторная работа №2 Однослойный перцептрон и его обучение методом дельта-правила.....	9
Лабораторная работа №3 Кластеризация алгоритмом к-средних	13
Лабораторная работа №4 Оптимизация функций с помощью генетических алгоритмов	16
Лабораторная работа №5 Обучение агентов методом искусственной жизни. Создание агента с нейроконтроллером.	20
Лабораторная работа №6 Обучение агентов методом искусственной жизни. Создание окружающей среды.	22
Лабораторная работа №7. Обучение агентов методом искусственной жизни. Основной цикл моделирования.	23
Лабораторная работа №8. Обучение агентов методом искусственной жизни. Графическая реализация, сборка и запуск.	24
Список литературы	25

Лабораторная работа №1 Экспертные системы

Цель работы: разработать экспертную систему с базой знаний на основе одной из классических моделей – продукционной, фреймовой, семантической.

Теоретический материал:

1. Продукционная модель

Продукционная модель представляет собой форму хранения знаний в формате правил, где каждое правило (также называемое продукцией) представлено в следующем виде:

Правило 1:

ЕСЛИ (условие)

ТО (заключение)

Множество таких правил, описывающих некоторую предметную область, представляет собой т.н. **Базу знаний** экспертной системы.

Например, в базе знаний метеорологической ЭС будут храниться такие правила:

Правило 1

Если (дует западный ветер) И (падает давление)

ТО (будет дождь)

Правило 2

Если (ночью был дождь) И (температура ночью опустилась ниже нуля)

ТО (будет гололёд)

Подобные правила накапливаются в базе знаний и затем обрабатываются механизмом логического вывода экспертной системы. Общая схема логического вывода экспертной системы показана на рис. 1.

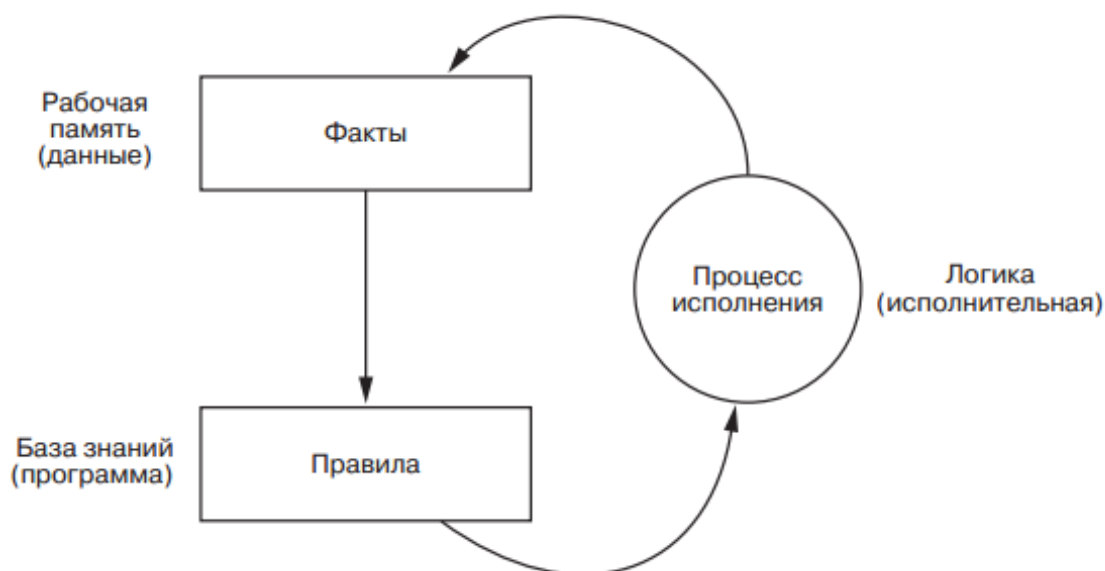


Рисунок 1 – Механизм рассуждения экспертной системы

В данном варианте, нужно сделать экспертную систему с БЗ на 15+ правил для выбранной предметной области. Программа должна рассуждать, отвечать на вопросы по предметной области. Например «Какая завтра будет погода?» при известных факторах окружающей среды. Правила должны храниться либо в коде программы в формате String, либо во внешнем текстовом файле и подгружаться программой. Язык программирования использовать любой. Выполнение в виде консольного приложения нежелательно.

2. Семантическая сеть.

Семантическая сеть представляет собой сеть, где объекты связаны друг с другом смысловыми отношениями. Пример такой сети показан на рисунке 2. Здесь мы видим семантическую сеть, связывающую с собой по смыслу самолёты с птицами.

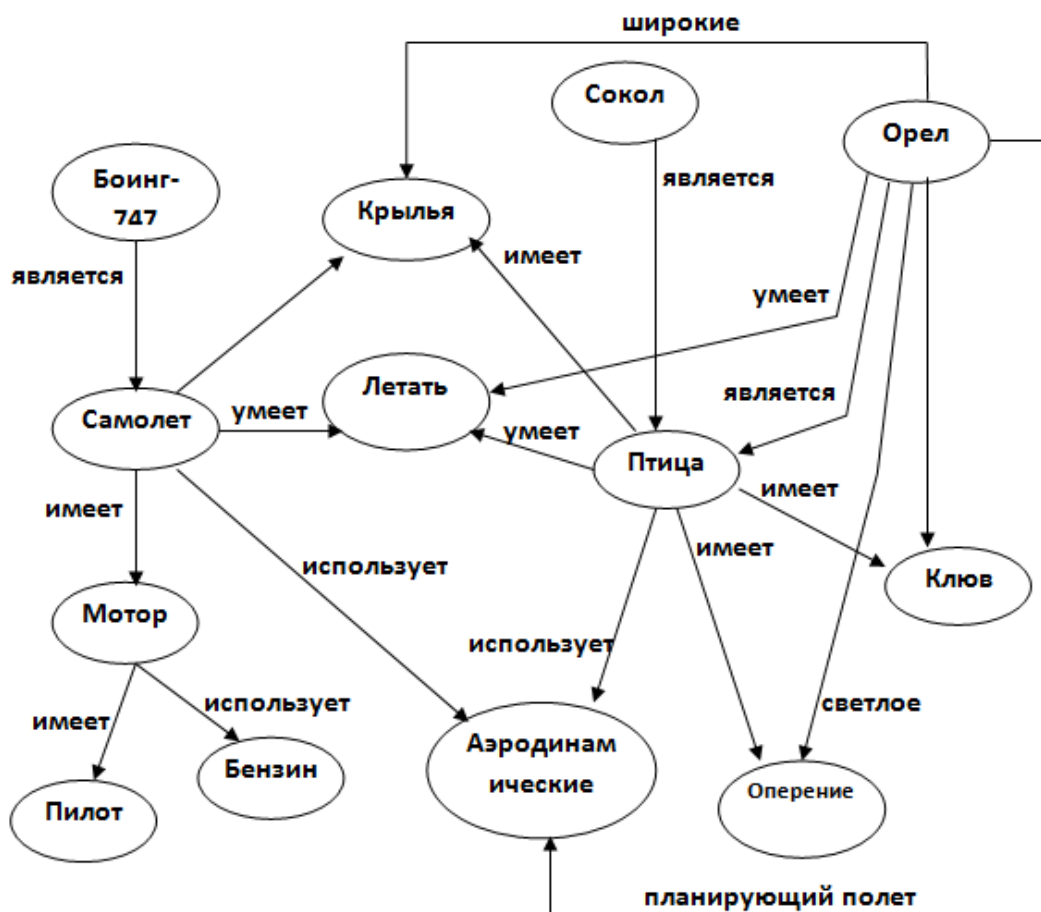


Рисунок 2 – Классифицирующая семантическая сеть

По построенной сети программа позволяет:

- получить все узлы, связанные с отдельным узлом конкретным соотношением;
- получить имена всех отношений для отдельного узла;
- получить все пары узлов, связанных конкретным соотношением;
- проследить путь от одного узла к другому.

В данном варианте нужны придумать свою семантическую сеть на 15+ объектов, запрограммировать её и создать программу, рассуждающую по сети и способную отвечать на вопросы пользователя что связано с чем и каким образом. Наиболее рационально делать семантическую сеть на основе взвешенного графа. Язык программирования использовать любой. Выполнение в виде консольного приложения нежелательно.

3. Фрейм

Фрейм – это минимально возможное описание сущности какого-либо события, ситуации, процесса или объекта. В историческом плане развитие фреймовой модели связано с теорией фреймов М. Минского, определяющей способ формализации знаний, используемый при решении задач распознавания образов (сцен) и понимания речи. «Отправным моментом для данной теории служит тот факт, что человек, пытаясь познать новую для себя ситуацию или по-новому взглянуть на уже привычные вещи, выбирает из своей памяти некоторую структуру данных (образ), называемую нами фреймом, с таким расчетом, чтобы путем изменения в ней отдельных деталей сделать ее пригодной для понимания более широкого класса явлений или процессов». Другими словами, *фрейм – это форма описания знаний, очерчивающая рамки рассматриваемого (в текущей ситуации при решении данной задачи) фрагмента предметной области.*

Модель фрейма является достаточно универсальной, поскольку позволяет отобразить все многообразие знаний о мире через:

- *фреймы-структуры, для обозначения объектов и понятий (заем, залог, вексель);*
- *фреймы-роли (менеджер, кассир, клиент);*
- *фреймы-сценарии (банкротство, собрание акционеров, празднование именин);*
- *фреймы-ситуации (тревога, авария, рабочий режим устройства) и др.*

В данном варианте нужно создать фрейм на 15+ элементов и сделать программу, способную рассуждать по этому фрейму, отвечать на вопросы пользователя что входит в состав чего. Наиболее рационально делать фреймы на основе модели ООП, где каждый фрейм - это класс или объект класса. Язык программирования использовать любой. Выполнение в виде консольного приложения нежелательно.

Пример фрейма на 15 объектов дан на рисунке 3.

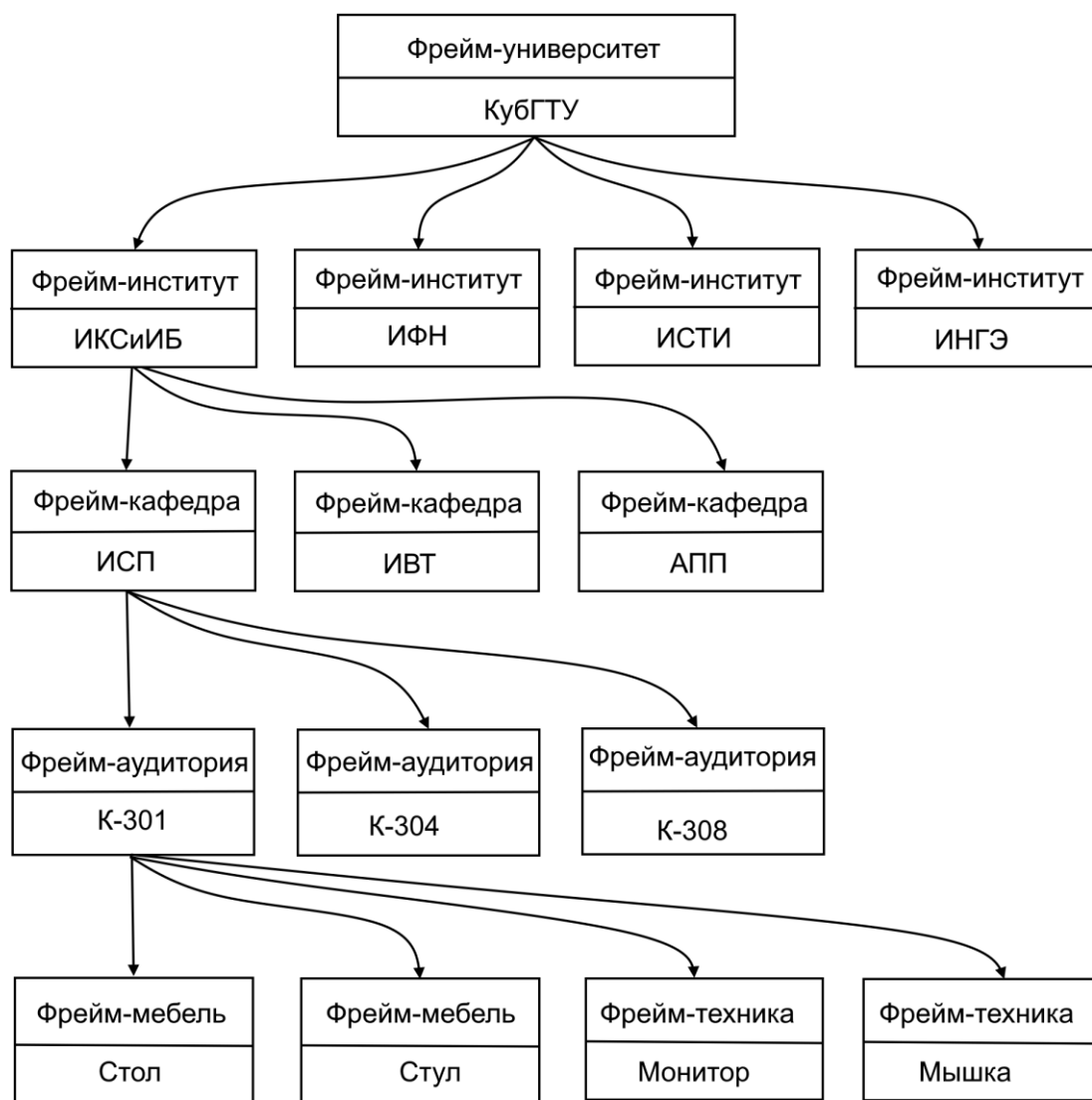


Рисунок 3 - Пример фрейма «Университет» на 15 элементов

Задание: разработать экспертную систему на основе одной из предложенных моделей (по варианту) с базой знаний из не менее 15 объектов.

Варианты:

1. Продукционная модель если ваш номер в группе 1,4,7,10,13,16,19,22,25,28
2. Семантическая сеть если ваш номер в группе 2,5,8,11,14,17,20,23,26,29
3. Фреймовая модель если ваш номер в группе 3,6,9,12,15,18,21,24,27,30

Лабораторная работа №2 Однослойный перцептрон и его обучение методом дельта-правила

Цель работы: запрограммировать и обучить однослойный перцептрон.

Теоретический материал:

Перцептрон – классическая и наиболее распространённая модель искусственной нейронной сети (ИНС), т.е. сети искусственных нейронов. Модель искусственного нейрона дана на рисунке 4. Сигналы, поступающие на вход нейрона, суммируются с учётом весовых коэффициентов w_i каналов x_i , $i = 1, \dots, n$, где n – размерность пространства входных сигналов. Потенциал нейрона определяется по формуле:

$$P = \sum_{i=1}^n w_i x_i .$$

Взвешенная сумма поступивших сигналов (потенциал) преобразуется с помощью функции активации $f(P)$ в выходной сигнал нейрона Y , который передается другим нейронам сети, т.е. $Y = f(P)$.

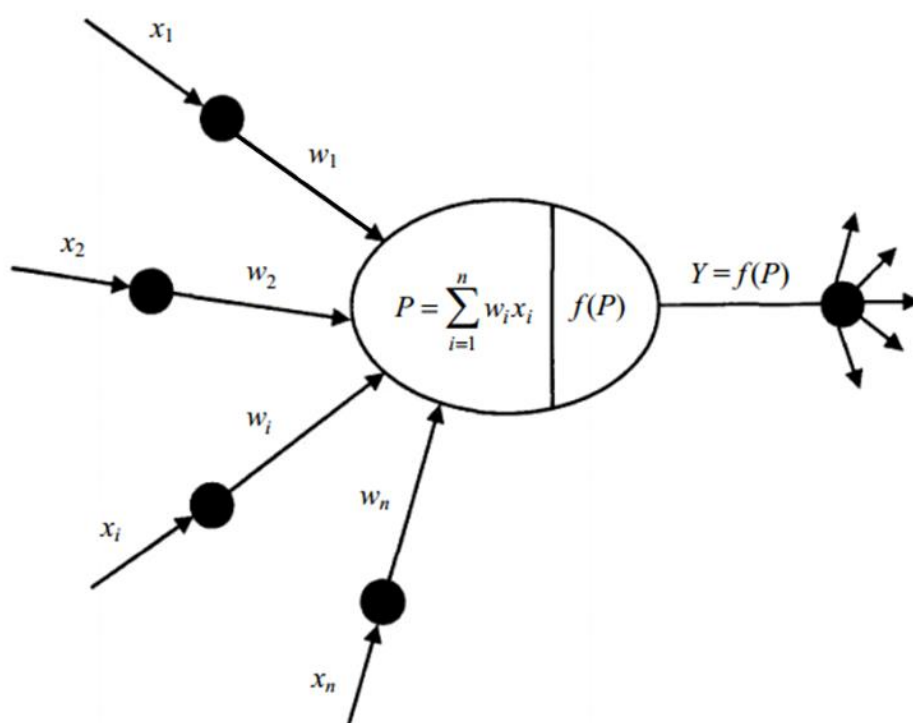


Рисунок 4 – Модель искусственного нейрона МакКаллока –Питтса

Вид функции активации является важнейшей характеристикой нейрона и в целом играет решающую роль в обучении ИНС и её последующей работе. Классический нейрон МакКаллока-Питтса предполагал

использование ступенчатой функции активации, что также вполне подходит для решения поставленной в данной ЛР задачи. Ступенчатая функция активации предельно проста для понимания и реализации: она выдаёт единицу если потенциал P превышает некоторый порог P^* . Иначе выдаёт ноль. Для данной ЛР вполне подойдёт ступенчатая функция с нулевым порогом.

$$Y = \begin{cases} 1 & \text{при } P \geq 0 \\ 0 & \text{при } P < 0 \end{cases}$$

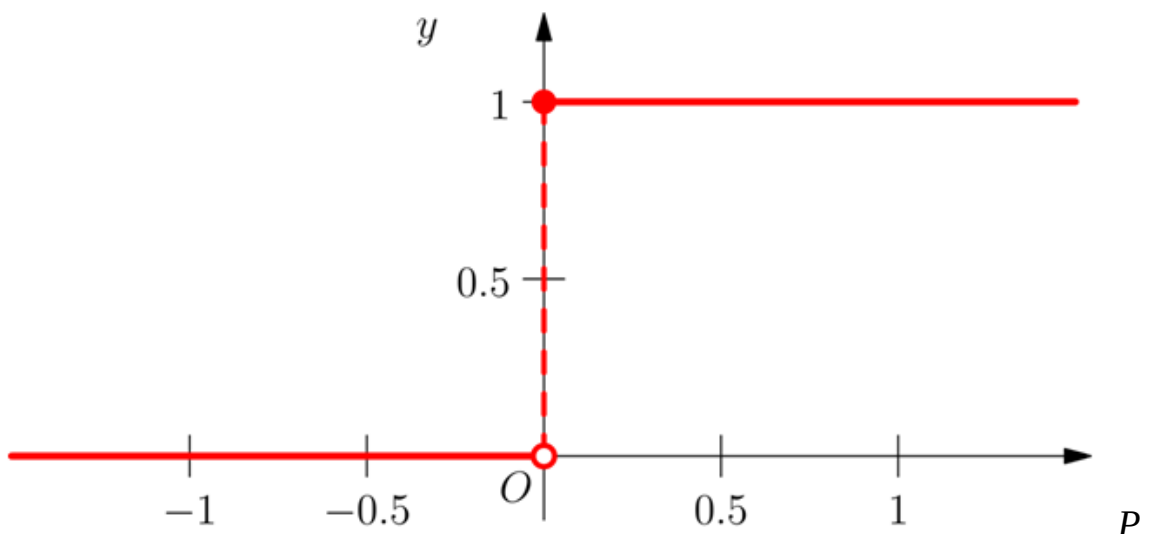


Рисунок 5 – Ступенчатая (пороговая) функция активации нейрона при $P^* = 0$

Порядок выполнения лабораторной работы:

1. Нарисовать в графическом редакторе картинки 30x30 пикселей, на каждой из которых изображена буква имени студента. Количество картинок – 5-7 штук.
2. Запрограммировать искусственный нейрон, принимающий на вход все пиксели картинки в формате: 1 если чёрный пиксель, 0 если белый. Для считывания цвета пикселя в C# есть команда `GetPixel` у объекта класса `Bitmap`. Загрузить картинку из файла можно командой `Image.FromFile()`. Наиболее оптимальный вариант программирования нейрона – как отдельный класс, в который инкапсулированы: веса входных каналов, значения входных сигналов, функция активации и значения выходного сигнала. Тогда слой нейронов можно хранить в виде коллекции объектов класса нейрон.
3. Создать столько нейронов, сколько букв в алфавите. Таким образом для распознавания букв русского алфавита на картинках 30x30

пикселей, вам понадобится 33 нейрона, каждый из которых принимает на вход 900 сигналов, т.е. по одному сигналу от каждого пикселя (1-если чёрный пиксель, 0 – если белый). Архитектура требуемой сети дана на рисунке 6.

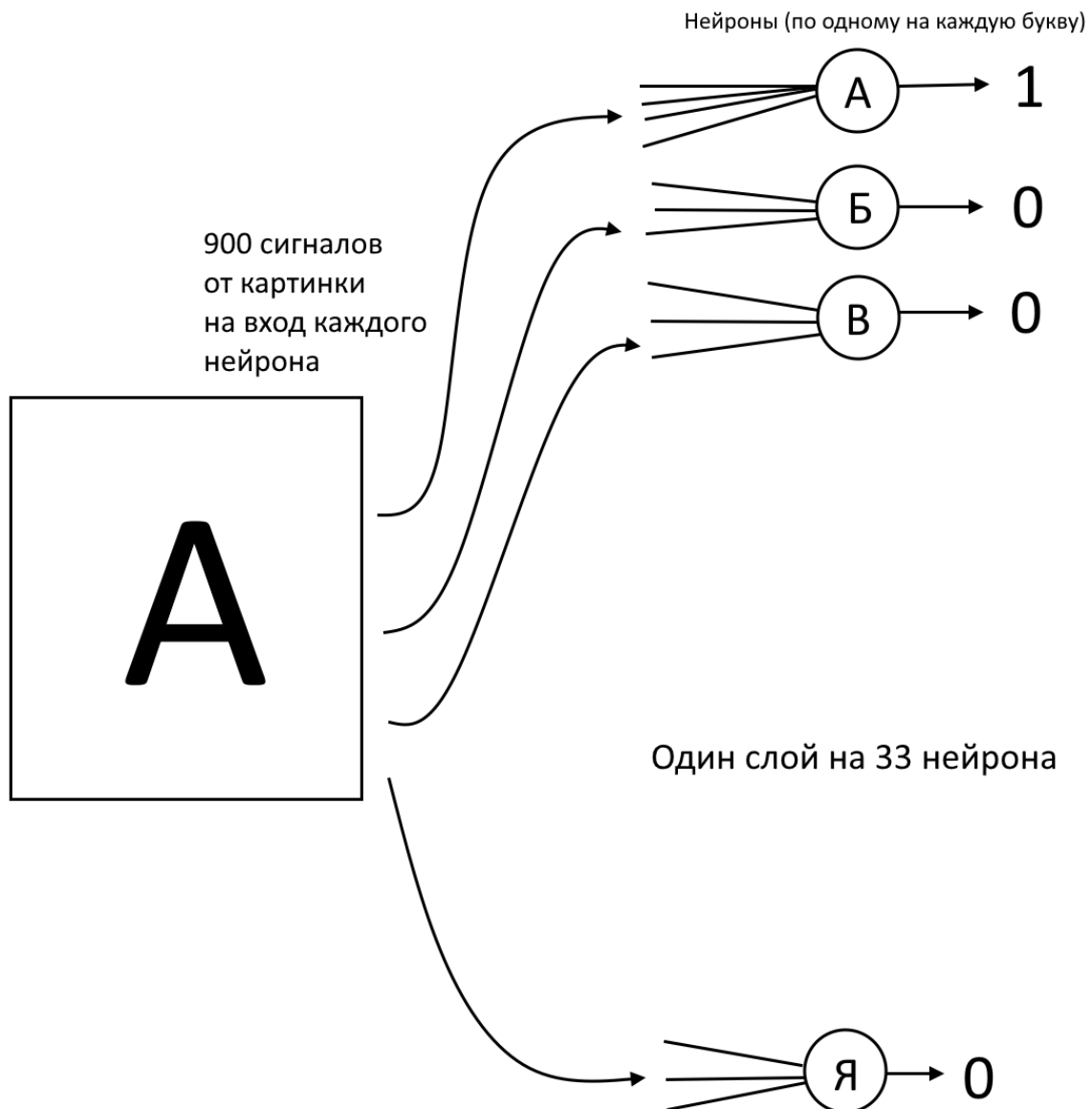


Рисунок 6 – Структура однослойного перцептрона для распознавания букв русского алфавита

4. Задачей нейросети в таком случае является, чтобы каждый нейрон выдавал на выходе 1 если на картинке показана его буква и 0 если не его.

5. Подготовить вектора эталонных ответов для каждой картинки. Для буквы А это вектор из 33 выходных сигналов, где первый сигнал равен единице (потому что при виде буквы А только соответствующий ей нейрон должен выдать 1), а все последующие сигналы – нули. Для буквы Б – только второй сигнал единичный. И.т.д.

6. Взять первую букву и начать обучение сети. Для этого подать входные сигналы на вход сети. Снять выходные сигналы.

7. Вычислить значения ошибок нейронов по формуле:

$$E = D - Y,$$

где E – ошибка нейрона,

D – эталонное значение,

Y – фактический выходной сигнал нейрона.

8. Перевычислить все веса всех нейронов по формуле:

$$w_{ij}(t + 1) = w_{ij}(t) + \eta * E_i * x_j$$

Где $w_{ij}(t)$ – вес j -го канала у i -го нейрона на эпохе t ,

E_i – ошибка i -го нейрона,

x_j – входной сигнал на j -м канале,

η – константа скорости обучения (принять любую от 0.05 до 0.2).

9. Взять другую букву и повторить пункты 6-8. Повторять для всех букв.

10. Создать программу с графическим интерфейсом, где подгружаются картинки для последующего обучения или распознавания. Результаты работы ИНС должны выводиться в список, например, ListBox. После каждого применения ИНС в список должны выводиться все буквы, нейроны которых выдали 1. По мере обучения количество букв в списке закономерно будет уменьшаться.

11. Конечным результатом обучения ИНС является ситуация, когда при подаче картинки, в список добавляется только одна буква – соответствующая той, что нарисована на картинке.

Пример интерфейса полученной программы дан на рисунке 7.

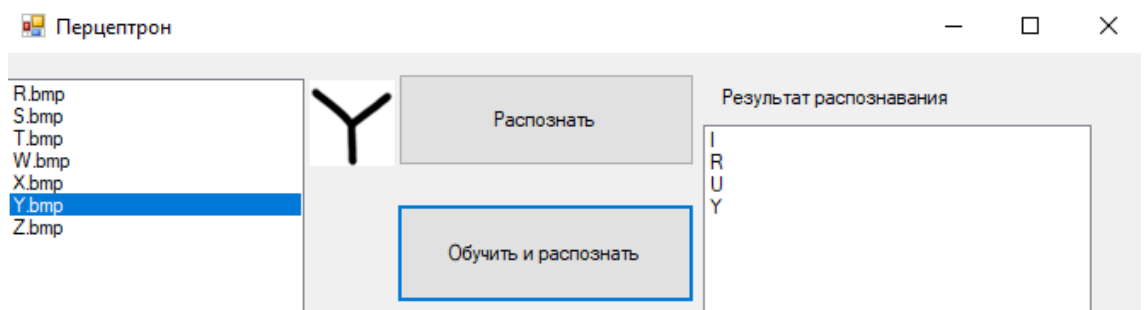


Рисунок 7 – Программа Перцептрон

Задание: запрограммировать и обучить однослойный перцептрон распознавать буквы своего имени на картинках. Использовать любой язык программирования. Консольное приложение нежелательно.

Лабораторная работа №3 Кластеризация алгоритмом k-средних

Цель работы: Реализовать программно кластеризацию алгоритмом k-средних.

Теоретический материал:

Алгоритм k-средних

Определение: *Центроид* — точка, которая является центром кластера.

k-средних (k-means) — наиболее популярный метод кластеризации. Алгоритму широко отдается предпочтение из-за его простоты реализации, большой скорости (а это очень важно при работе с видео).

Действие алгоритма таково, что он стремится минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров. Проще говоря, это итеративный алгоритм, который делит данное множество пикселей на k кластеров, точки которых являются максимально приближенными к их центрам, а сама кластеризация происходит за счет смещения этих же центров.

Также следует оговорить то, что метод k-средних очень чувствительный к шуму, который может существенно исказить результаты кластеризации. Так что в идеале, перед кластеризацией, нужно прогнать кадры через фильтры, предназначенные для его уменьшения.

Принцип кластеризации методом k-средних:

1. Надо выбрать из множества k пикселей те пиксели, которые будут центроидами соответствующих k кластеров.
2. Выборка начальных центроидов может быть как случайной, так и по определенному алгоритму.
3. Входим в цикл, который продолжается до тех пор, пока центроиды кластеров не перестанут изменять свое положение.
4. Обходим каждый пиксель и смотрим, к какому центроиду какого кластера он является ближайшим.
5. Нашли ближайший центроид? Привязываем пиксель к кластеру этого центроида.
6. Перебрали все пиксели? Теперь нужно вычислить новые координаты центроидов k кластеров.

Теперь проверяем координаты новых центроидов. Если они соответственно равны предыдущим центроидам — выходим из цикла, если нет, возвращаемся к пункту 3.

На рисунке 8 показан результат метода к-средних в общем виде

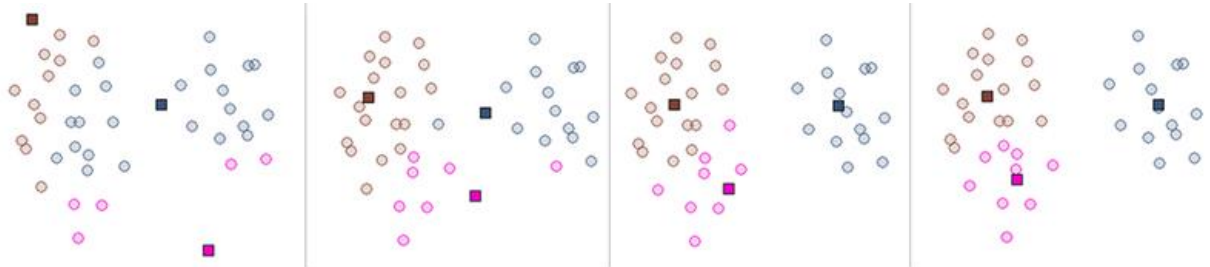


Рисунок 8 – Результат кластеризации по к-средних в общем виде

Для прорисовки можно использовать PictureBox в C#.

Код настройки:

1. Подключаем using System.Drawing;
2. В глобальном поле объявляем Graphics g;
3. В функции прорисовки вызываем `g = pictureBox1.CreateGraphics();`
4. Чтобы очистить поле вызываем `g.Clear(Color.White);`
5. Чтобы нарисовать объект по координатам x,y в виде круга с радиусом w
вызываем `g.DrawEllipse(new Pen(Brushes.Black), x, y, w, w);`

Если неудобен PictureBox, можно использовать, например, DataGridView, как это показано на рисунке 9.

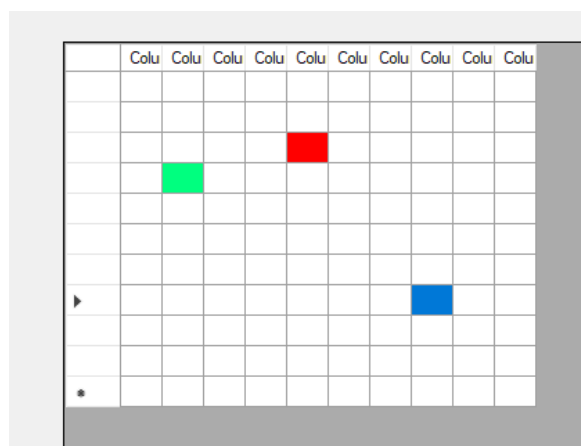


Рисунок 9 – Использование DataGridView в C# для визуализации объектов на координатной сетке

Настроить DataGridView можно следующим кодом:

Код для добавления 10 столбцов и 10 строк и установления ширины колонки:

```
for (int i = 0; i < 10; i++)
{
    dataGridView1.Columns.Add(i.ToString(), "");
    dataGridView1.Rows.Add();
    dataGridView1.Columns[i].Width = 30;
}
```

Закрашивание клетки по координатам x, y красным цветом:

```
dataGridView1.Rows[y].Cells[x].Style.BackColor = Color.Red;
```

Задание: Реализовать программно кластеризацию алгоритмом к-средних для поля из случайно сгенерированных точек. Сделать графическую прорисовку процесса в программе.

Лабораторная работа №4 Оптимизация функций с помощью генетических алгоритмов

Цель работы: Реализовать программно оптимизацию функции на основе генетических алгоритмов.

Теоретический материал:

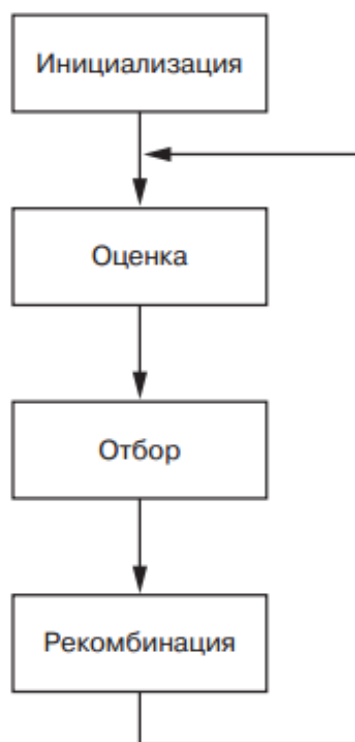


Рисунок 10 – Общая схема работы генетического алгоритма

Выполнение генетического алгоритма включает три основных шага (причем для каждого шага предусмотрен большой разброс возможных вариантов). Эти шаги показаны на рисунке 10.

Генетический алгоритм выполняется в три этапа (если не учитывать начальное создание популяции). Во время оценки определяется здоровье популяции. Далее производится отбор подгруппы хромосом на основании предварительно заданного критерия. Наконец, выбранная подгруппа рекомбинируется, в результате чего получается новая популяция.

Теперь рассмотрим поиск экстремума функции с помощью генетического алгоритма. В этом примере выполняется поиск параметров, которые позволяют максимизировать уравнение:

$$f(x,y) = \frac{1}{1+x^2+y^2}$$

Это уравнение графически представлено на рисунке 11 (трехмерный график и контурный график)

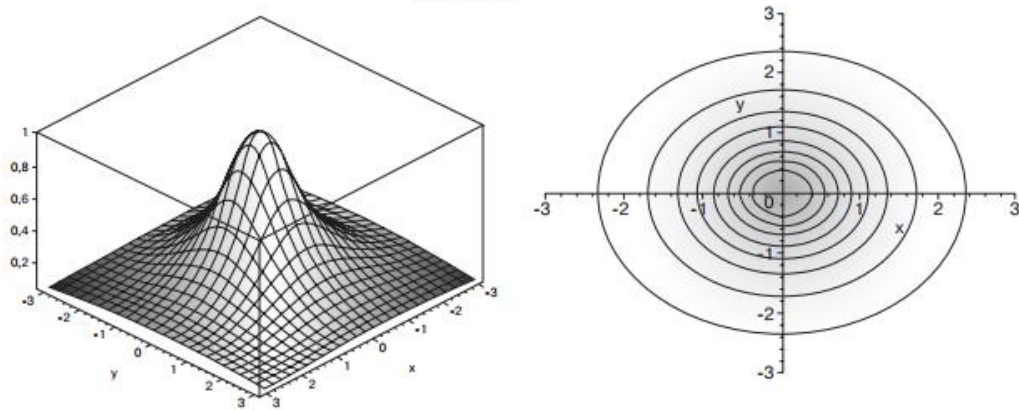


Рисунок 11 – Трёхмерный и контурный график работы генетического алгоритма

Пример полученной программы дан на рисунках 12-14. Здесь программно производится оптимизация предложенной функции ($z=1/(1+x*x+y*y)$).



Рисунок 12 – Первый шаг генетической оптимизации функции

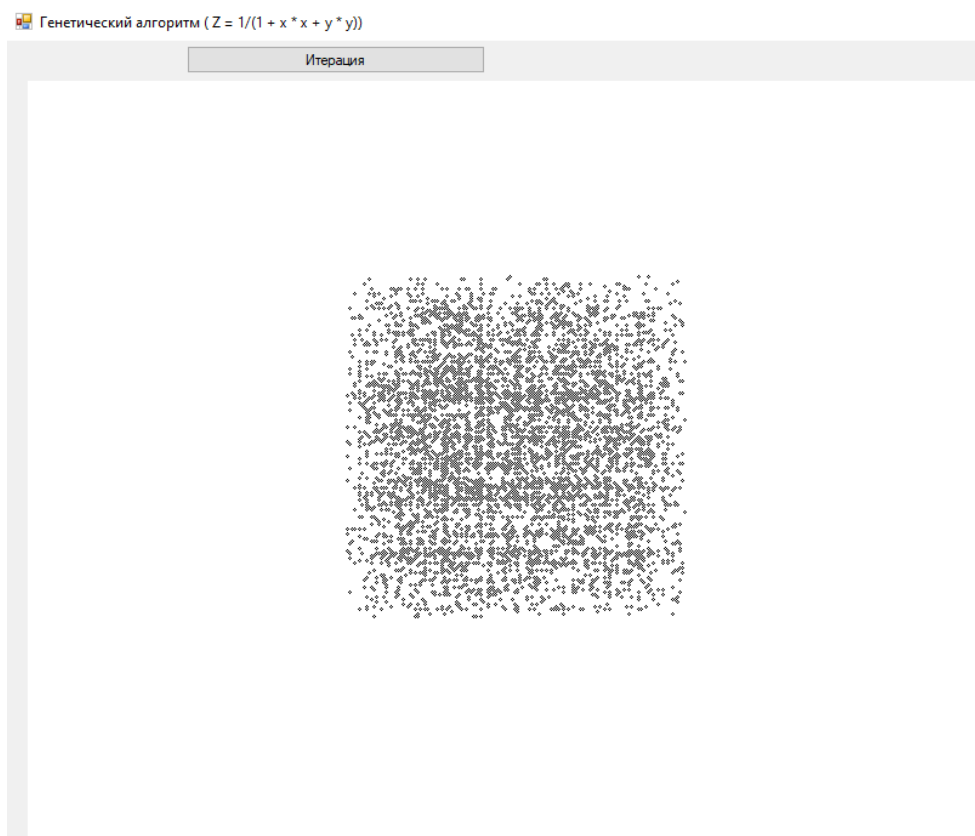


Рисунок 13 – Второй шаг генетической оптимизации функции

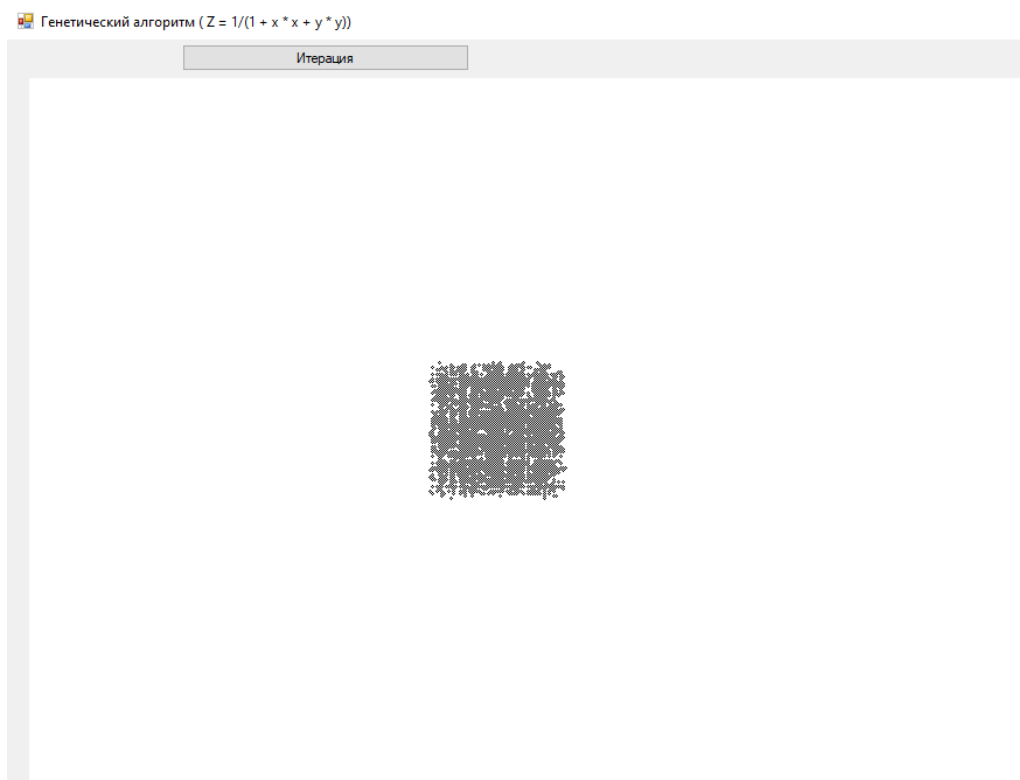


Рисунок 14 – Третий шаг генетической оптимизации функции

Как видим на рисунках, особи стремятся к центру поля, потому что максимизация функции идёт именно в этом направлении.

Задание: Реализовать программно поиск экстремумов на основе генетических алгоритмов. Выбрать любую функцию двух аргументов и реализовать её генетическую оптимизацию с графической прорисовкой расположения особей (вид сверху, из оси Z).

Лабораторная работа №5 Обучение агентов методом искусственной жизни. Создание агента с нейроконтроллером.

Цель работы: в рамках четырёх работ (ЛР5-8), реализовать обучение интеллектуальных нейросетевых агентов методом искусственной жизни. Цель ЛР №5 – создать класс агента с нейроконтроллером.

Теоретический материал:

Искусственная жизнь представляет собой виртуальный мир, где агенты взаимодействуют друг с другом и борются за выживание. Агентом в данном случае является действующее лицо, способное совершать следующие действия: поворот влево, поворот вправо, шаг вперёд, поедание. Для принятия решения агент анализирует окружающую обстановку с помощью однослойной ИНС без функции активации. Архитектура сети показана на рисунке 15.

Для выполнения данной ЛР вполне подойдёт код, уже созданный в ЛР2. Нейросеть агента будет состоять из четырёх нейронов с 12 входами у каждого. Функция активации – линейная: $y=kr$, где p – потенциал нейрона, $k=1$.

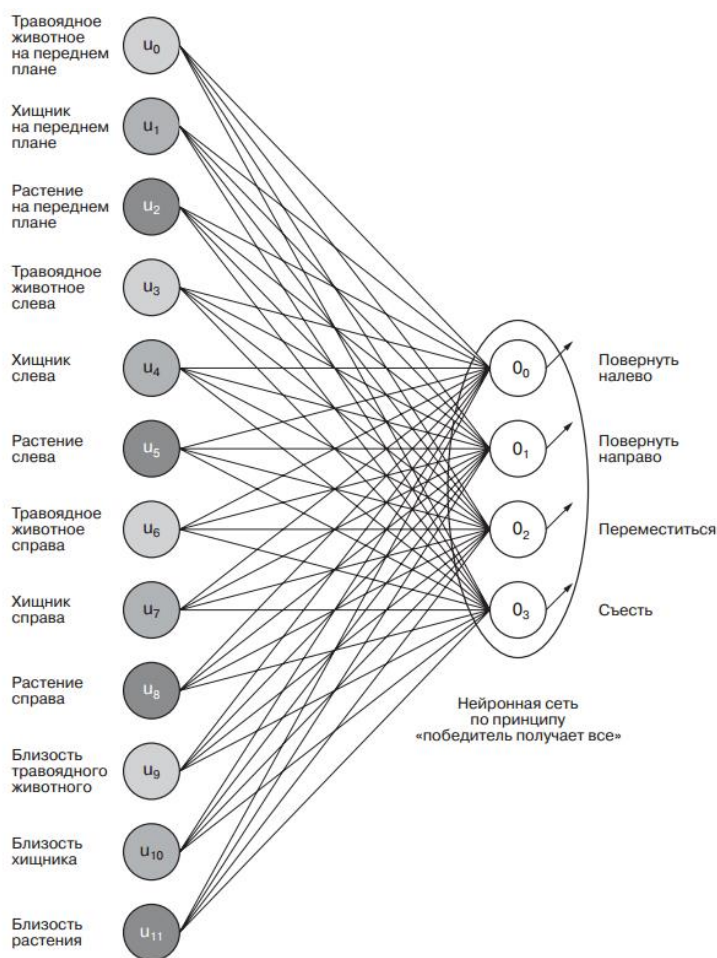


Рисунок 15 – Архитектура ИНС агента

Помимо нейросети управления, у агента также должен быть параметр здоровья, а также пространственные координаты x , y .

Задание: запрограммировать класс агента с нейроконтроллером.

Общее задание на ЛР 5-8:

Если ваш вариант **нечётный**, то ваши агенты: хищники, травоядные, растения. Хищники поедают только травоядных, травоядные поедают только растения, растения генерируются случайно и существуют, пока их кто-то не съест.

Если **чётный**, то вместо хищников у вас - фагоциты, вместо травоядных – бактерии, вместо растений – молекулы глюкозы. Бактерии могут есть только глюкозу. Фагоциты могут есть и бактерий, и глюкозу, но глюкоза даёт энергии в четыре раза меньше. Глюкоза генерируется случайно и существует, пока её не съедят.

Вариант определяется вашим номером в группе по списку.

Лабораторная работа №6 Обучение агентов методом искусственной жизни. Создание окружающей среды.

Цель работы: Создание среды, в которой будет протекать искусственная жизнь.

Теоретический материал:

Искусственная жизнь протекает на координатной сетке тороидной формы, т.е. если агент пересекает правый край мира, он появляется у левого края. Аналогично по вертикали.

Также, агенты должны иметь возможности навигации в этом мире посредством восприятия непосредственно прилегающей к ним зоны. Пример зоны восприятия дан на рисунке 16.

	Фронт	Фронт	Фронт	Фронт	Фронт	
	Слева	Близость	Близость	Близость	Справа	
	Слева	Близость	Агент	Близость	Справа	

Рисунок 16 – Область предчувствия агента. Ориентация на север

Задание: разработать тороидный двухкоординатный мир и снабдить агентов из предыдущей ЛР функциями восприятия близлежащей к ним области.

Лабораторная работа №7. Обучение агентов методом искусственной жизни. Основной цикл моделирования.

Цель работы: Реализовать основной цикл, обновляющий все процессы искусственной жизни.

Теоретический материал: рекомендуется использовать компонент таймер, доступный в MS Visual Studio. В основном таймере нужно запрограммировать следующие операции:

1. Обработка зон восприятия всех агентов, подача им на вход ИНС всех новых сигналов
2. Просчёт всем агентам реакции их ИНС
3. Выполнение принятых решений всех агентов (поедание одним агентом другого возможно только в случае совпадения двух факторов: 1. ИНС этого агента выдала сигнал на поедание. 2. В зоне близости находится еда)
4. Обновление здоровья всех агентов, уничтожение умерших
5. Размножение агентов, чьё здоровье выше порога (Создание нового агента-потомка, мутация его ИНС, передача ему половины здоровья родителя)
6. Генерирование новых растений (или глюкозы)

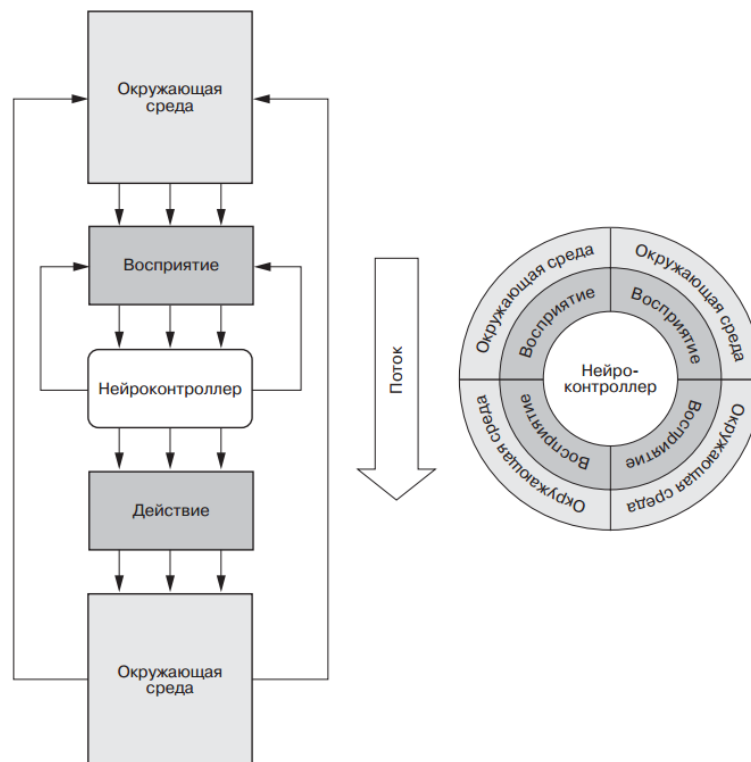


Рисунок 17 - Основной цикл (поток) жизни агента

Задание: реализовать основной поток моделирования искусственной жизни

Лабораторная работа №8. Обучение агентов методом искусственной жизни. Графическая реализация, сборка и запуск.

Цель работы: собрать вместе все компоненты среды и запустить.

Теоретический материал: Для графической реализации можно использовать элемент PictureBox доступный в C#. Агентов можно изображать символами по координатам с помощью функции DrawString. Пример полученной программы дан на рисунке 18. На рисунке зелёные агенты – растения, синие – травоядные, красные – хищники.

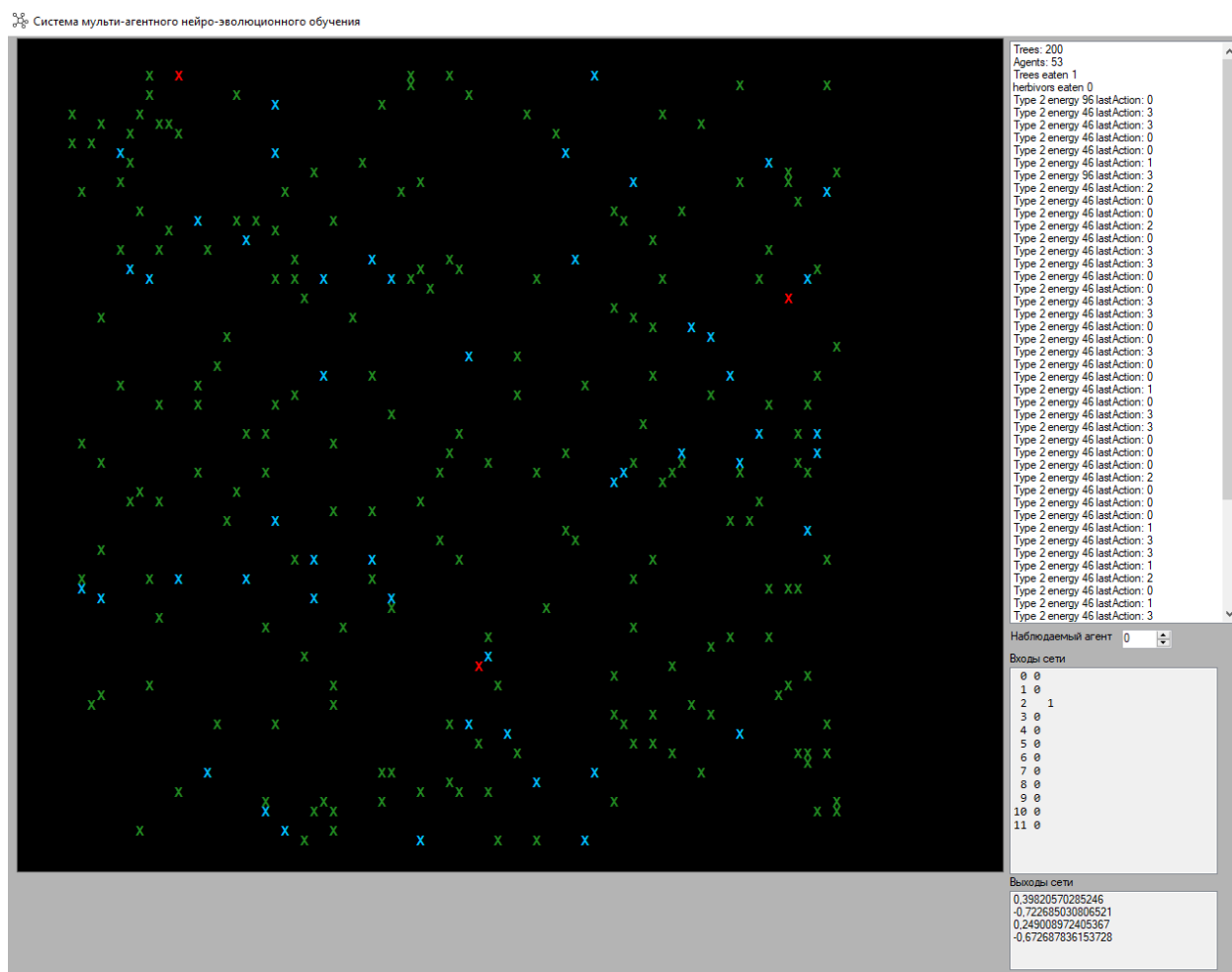


Рисунок 18 – Программа Искусственная жизнь

Если неудобен PictureBox, можно использовать DataGridView. Инструкция дана в предыдущих ЛР.

Задание: завершить программную реализацию искусственной жизни на основе ЛР 5-8.

Список литературы

1. Основы искусственного интеллекта [Электронный ресурс] / Боровская Е.В., Давыдова Н.А., - 3-е изд., (эл.) - М.:Лаборатория знаний, 2016. - 130 с.: ISBN 978-5-00101-421-8 - Режим доступа: <http://znanium.com/catalog/product/366789>
2. Автономный искусственный интеллект [Электронный ресурс] / Жданов А.А., - 4-е изд., (эл.) - М.:БИНОМ. ЛЗ, 2015. - 362 с.: ISBN 978-5-9963-2540-5 - Режим доступа: <http://znanium.com/catalog/product/539035>
3. Джонс М. Т. Программирование искусственного интеллекта в приложениях [Электронный ресурс] / М. Тим Джонс ; Пер. с англ. Осипов А. И. - М. : ДМК Пресс, 2011. - 312 с.: ил: <http://znanium.com/bookread2.php?book=410211>